Milestone 1: Regret, CFR and Best Response on Kuhn Poker

holdem-cfr walkthrough

The problem CFR solves

In a game of imperfect information, a good strategy cannot be a fixed rule like "bet the king". An opponent who learns the rule exploits it. Instead, a strategy is a probability distribution over actions at every decision point, and the target is a *Nash equilibrium*: a pair of strategies such that neither player can gain by changing theirs alone. In two-player zero-sum games an equilibrium strategy has a powerful guarantee. Against *any* opponent it earns at least the game value in expectation. You do not need to model the opponent at all.

The difficulty is that a player does not know which exact situation they are in. In poker you see your own cards but not your opponent's. The situations you cannot tell apart form an **information set**, and your strategy must assign the same distribution to every situation in it. Counterfactual regret minimization, introduced by Zinkevich et al. in 2007, finds equilibria by running a simple learning rule at every information set independently and letting them co-evolve.

Regret matching, the local learning rule

Imagine playing rock-paper-scissors repeatedly. After each round you ask, for each action, "how much better would I have done if I had played that instead?". That quantity is the action's *regret* for the round. It is the payoff the action would have earned minus the payoff you actually earned. You played rock, the opponent played paper, and you lost 1. Paper would have tied (0), so paper's regret is $0 - (-1) = 1$. Scissors would have won (+1), so scissors' regret is $1 - (-1) = 2$. Rock's regret is zero by definition.

Regret matching keeps a running sum of each action's regret and plays each action in proportion to its *positive* cumulative regret. After that one round the strategy is rock 0, paper 1/3, scissors 2/3. If no action has positive regret, it plays uniformly. Negative regrets are clipped because an action you regret *not* having played is evidence for it, while an action that did worse than your average is not evidence for anything. You simply stop playing it until circumstances change.

The key theorem is that regret matching's average regret per round shrinks like $1/\sqrt{T}$. In a two-player zero-sum game, if both players have average regret below ε, their *average* strategies form a 2ε-equilibrium. The word "average" matters. The strategy each player uses on a given round can cycle forever, and only the

time-average converges. That is why the solver tracks two things per action: cumulative regret, which drives what to play next, and cumulative strategy, which is what you ship.

```
pub fn regret_matching(regrets: &[f64], out: &mut [f64]) {
    let positive_sum: f64 = regrets.iter().map(|&r| r.max(0.0)).sum();
    if positive_sum > 0.0 {
        for (o, &r) in out.iter_mut().zip(regrets) {
            *o = r.max(0.0) / positive_sum;
        }
    } else {
        out.fill(1.0 / out.len() as f64);
    }
}
```

The rock-paper-scissors example above is literally a unit test in `regret.rs`. A property test checks, on thousands of random regret vectors, that the output always sums to one and never puts weight on a non-positive-regret action unless every action is non-positive.

From one decision to a whole game: counterfactual values

Poker has many decision points in sequence, and payoffs only arrive at the end. CFR's insight is to give each information set its own regret-matching learner and feed it the right signal. That signal is the **counterfactual value**: the expected payoff of reaching this information set, weighted by the probability that chance and the opponent would bring play here, and assuming that *you* had tried to get here.

The second half of that definition is the clever part. Your own probability of reaching the information set is deliberately left out. If it were included, an information set your current strategy rarely visits would get a tiny learning signal and would never improve, even though a rarely visited spot could matter a great deal once the rest of your strategy changes.

Concretely, a recursive traversal carries two reach probabilities down the tree. `reach_me` is the traversing player's own probability of having played to this node. `reach_other` is chance's probability times the opponent's. At the traverser's own decision node, the code computes the value of each action, combines them into the node value under the current strategy, and then does two updates:

```
for a in 0..children.len() {
    // Counterfactual regret: weighted by everyone else's reach.
    self.instant[offset + a] += reach_other * (action_values[a] - v);
    // Average strategy: weighted by the traverser's own reach.
    self.strategy_sum[offset + a] += self.weight * reach_me * self.current[offset + a];
}
```

The asymmetry is the whole algorithm in two lines. Regret is weighted by the *opponent's and chance's* reach, which is the counterfactual part. The average strategy is weighted by *your own* reach, because an information set you rarely reach should contribute little to what your average strategy does there.

Two subtleties that are easy to get wrong

The first concerns **snapshots**. An information set contains many histories. In Kuhn poker the set "I hold the king, nothing has happened yet" contains two histories, one per possible opponent card. A full traversal visits all of them. If regrets were updated in place as each history is visited, the second visit would see a different strategy from the first, and the iteration would no longer correspond to the algorithm the theory analyses. The solver therefore recomputes the current strategy for every information set once, before each player's traversal. It accumulates the traversal's regrets into a separate `instant` buffer and folds that buffer into the cumulative regrets only after the traversal finishes.

The second concerns **alternating updates**. Each iteration first updates player 0's regrets against the current profile. It then recomputes strategies and updates player 1's regrets against player 0's *new* strategy. This is a small change from the original simultaneous version, but it reliably converges faster, and CFR+ requires it, so every variant in this codebase uses it.

The Game trait

The engine talks to games only through a trait. A trait in Rust is an interface: a set of methods a type promises to provide. This trait also uses *associated types*. Each game picks its own concrete `State`, `Action` and `InfoKey` types, and the engine code is generic over them.

```
pub trait Game {
    type State: Clone;
    type Action: Copy + Debug + Eq;
    type InfoKey: Clone + Eq + Hash + Debug;

    fn root(&self) -> Self::State;
    fn node_type(&self, state: &Self::State) -> NodeType;   // Terminal | Chance | Decision(p)
    fn payoff(&self, state: &Self::State) -> f64;            // to player 0
    fn chance_outcomes(&self, state: &Self::State) -> Vec<(f64, Self::State)>;
    fn actions(&self, state: &Self::State) -> Vec<Self::Action>;
    fn apply(&self, state: &Self::State, action: Self::Action) -> Self::State;
    fn info_key(&self, state: &Self::State) -> Self::InfoKey;
}
```

The bounds after the colons, such as `Clone + Eq + Hash`, are promises the engine relies on. `InfoKey` must be hashable because tree construction groups histories into information sets with a hash map. When the engine is compiled against `Kuhn`, Rust

generates a specialized copy of every generic function for Kuhn's concrete types. This is called *monomorphization*, and it means the abstraction costs nothing at run time. There is no virtual dispatch and no boxing.

Two design choices are worth knowing. Chance outcomes return the *next states* directly rather than chance actions, which spares every game from inventing an action type for "deal the queen". States are values: `apply` returns a new state rather than mutating one. That makes recursive traversal trivially correct. It is too slow for hold'em, where we will switch to sampling and probably an undo-based interface, but the exact algorithms never see the trait anyway.

Compiling the game into a tree

Kuhn and Leduc are small enough to expand completely. `Tree::build` walks the game once and produces a flat arena: a `Vec<Node>` where each node refers to its children by index, plus a table of information sets. Each information set records its player, the list of tree nodes belonging to it, and an `offset` into flat per-action arrays. If information set 7 has three actions starting at offset 19, its regrets live at `regrets[19..22]`. Every exact algorithm then runs on plain indices with no hashing.

Nodes are stored in *pre-order*: a node's slot is reserved before its children are expanded, so a parent always has a smaller index than its children. Best response uses this to compute reach probabilities with a single forward sweep. The builder also asserts that every history in an information set offers identical legal actions. If a game's `info_key` accidentally hid information that determines the legal moves, the build fails loudly instead of producing a silently wrong strategy.

```
pub enum Node {
    Terminal { payoff: f64 },
    Chance { children: Vec<usize>, probs: Vec<f64> },
    Decision { player: usize, infoset: usize, children: Vec<usize> },
}
```

This is a Rust *enum with data*. Each variant carries its own fields, and `match` forces every use site to handle all three cases. If we add a variant later, the compiler lists every place that needs updating.

A note on borrowing: the Walk struct

The CFR traversal needs to *read* the tree and the current strategy while *writing* the regret and strategy-sum buffers. If all of these lived in one struct and the recursive function took `&mut self`, Rust would object. You cannot hold a shared reference into `self.current` while `self` is mutably borrowed. The idiomatic fix is to split the borrows up front into a small struct whose fields borrow different parts of the solver:

```
struct Walk<'a> {
    tree: &'a Tree,              // shared, read-only
    current: &'a [f64],          // shared, read-only
    strategy_sum: &'a mut [f64], // exclusive
    instant: &'a mut [f64],      // exclusive
    player: usize,
    weight: f64,
}
```

The borrow checker can see that `current` and `strategy_sum` are disjoint fields of the solver, so creating this struct is legal, and inside the recursion every access is unambiguous. The `'a` is a *lifetime*. It says that every reference in `Walk` lives at least as long as the traversal, which is how the compiler proves no reference outlives the data it points to. This "split the borrows into a context struct" pattern will come back when we parallelize MCCFR.

Exact best response and exploitability

To measure how close a strategy is to equilibrium, we compute how much a perfect adversary could win against it. A best responder for player i knows the opponent's strategy exactly but still cannot see the opponent's cards. So at each of its information sets it must pick *one* action for all the histories in that set at once. It chooses the action that maximizes the sum, over histories h in the set, of $\pi_{-i}(h) \times v(h \cdot a)$, where π_{-i} is chance-times-opponent reach. Computing best response per history instead of per information set is the classic bug here. It produces a "best response" that peeks at hidden cards, so its value is too high.

Values below an information set depend only on choices at deeper information sets, because Kuhn and Leduc have perfect recall. So a memoized recursion settles every choice exactly. Value lookups recurse down the tree, and each information set's best action is computed once and cached in an `Option<usize>`. **Exploitability** is then $(BR_0 + BR_1) / 2$. In a zero-sum game the game value appears with opposite signs in the two terms and cancels, so this quantity is non-negative and equals zero exactly at an equilibrium.

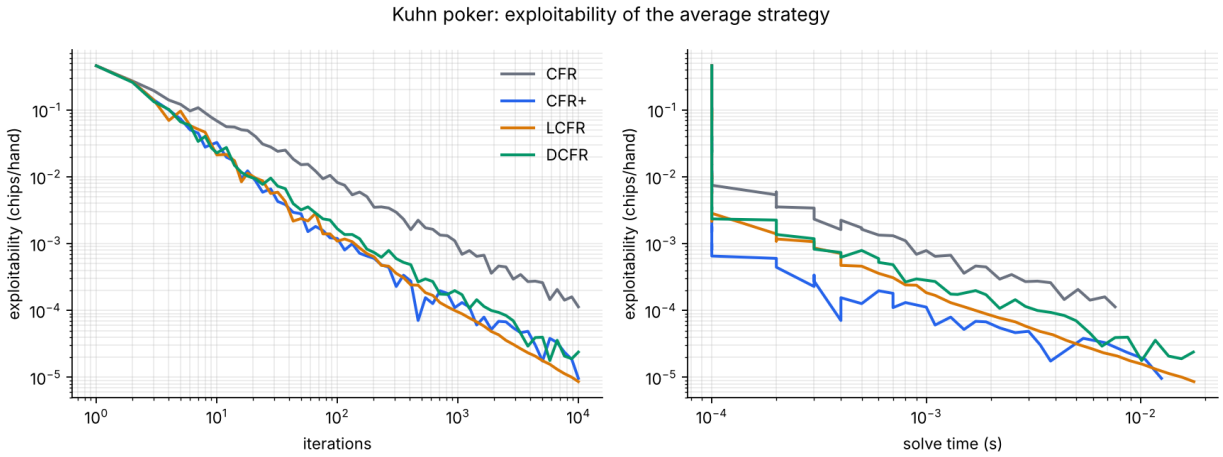
The tests, and what they prove

Kuhn poker has a known closed-form family of equilibria, parameterized by α in [0, 1/3]. Player 0 bets the jack with probability α, never bets the queen first, bets the king with probability 3α, and when checked-then-bet-into calls with the queen with probability α + 1/3. Player 1's strategy is fixed: bet the king, bluff the jack one third of the time after a check, call a bet with the queen one third of the time. The first test plugs this analytic strategy, at several values of α, straight into the best-response code and requires exploitability below 10^{-12} and a game value of exactly −1/18. That validates best response independently of CFR. The second test runs every CFR

variant for 20,000 iterations and requires the game value within 10^{-3} of $-1/18$ and exploitability below 10^{-3}.

The third test is the most convincing. CFR is allowed to pick any α, so the test reads off the α the solver chose and checks that the king-bet and queen-call probabilities obey the 3α and α + 1/3 relationships. A solver that merely got the value right by accident would fail it. Finally, a property test feeds hundreds of random strategy profiles into best response and checks that $BR_0 \geq EV$ and $BR_1 \geq -EV$ for every one, which is the defining property of a best response.

```
$ holdem-cfr solve --game kuhn --algo cfr --iters 100000 --print
55 nodes, 12 information sets (6 / 6)
game value (player 0): -0.055556
exploitability:        1.731e-5
  P0 J     Pass=0.797  Bet=0.203        alpha = 0.203
  P0 K     Pass=0.392  Bet=0.608        3 alpha = 0.609
  P0 Qpb   Pass=0.464  Bet=0.536        alpha + 1/3 = 0.536
  P1 Jp    Pass=0.667  Bet=0.333        bluff a third
  P1 Qb    Pass=0.667  Bet=0.333        call a third
```



Kuhn exploitability

Reading

Neller and Lanctot (2013), sections 2 and 3, cover exactly this milestone: regret matching on rock-paper-scissors, then CFR on Kuhn poker with the same counterfactual-reach reasoning. Zinkevich et al. (2007) has the original regret-decomposition theorem, which shows that the sum of per-information-set counterfactual regrets bounds overall regret. That theorem is the reason running independent learners at every information set is allowed at all.